SIE / ENTREGA FINAL / ANEXO B

Manual técnico

Arquitectura, modelo, instalación, seguridad y mantenimiento.

Control de documento	Detalle
Proyecto	MesaHelp · MVP académico de gestión de solicitudes
Versión del código	0.3.1 · candidata de cierre funcional (11/11/2026)
Tipo de entrega	Trabajo individual según asignación comunicada por docente
Autor / identificador	Hector Alejandro Garcés
Fecha	11 de noviembre de 2026
Estado	Validación local y GitHub Actions correctas; repositorio público

GitHub y GitHub Pages publicados. Instalación por tercero aún por verificar.

CORPORACIÓN UNIVERSITARIA AUTÓNOMA DEL CAUCA

Sistemas de Información Empresariales | Trabajo individual | Noviembre de 2026

Contenido

Contenido 2

1. Alcance y tecnologías 3

2. Modelo de datos y relaciones 3

3. Procesos, endpoints y módulos de código 4

Distribución del código 5

4. Instalación local y variables de entorno 5

4.1. Requisitos 5

4.2. Entorno Docker local 5

5. Seguridad y privacidad 6

6. Pruebas, respaldo y mantenimiento 6

6.1. Copias de seguridad 6

6.2. Mantenimiento / problemas conocidos 6

7. Repositorio, licencias y despliegue 6

1. Alcance y tecnologías

MesaHelp es un MVP académico de mesa de ayuda. Ofrece gestión de tickets, usuarios y sesiones, historial, acceso de invitado por código privado, avisos Live, explorador local y reportes con CSV. **No es un sistema de mesa de ayuda en producción certificado**.

Componente	Tecnología / función
Servidor	Node.js 22.13+ con módulo http nativo. Se verificó con Node 22.16.0 en el entorno de generación; en Windows se utilizó Node 22.19.0.
Base de datos	SQLite nativo node:sqlite (experimental en Node 22), WAL y claves foráneas.
Frontend	HTML renderizado por servidor, CSS propio, JavaScript nativo (explorer.js y live.js).
Live	WebSocket de una sola instancia, avisos unidireccionales; refresco HTTP autorizado.
CSV	src/reports.js; Excel con UTF-8 y comas.
Contenedor	Dockerfile Node 24 Alpine, usuario sin privilegios, healthcheck.
Dependencias	Ningún paquete npm externo en package.json.

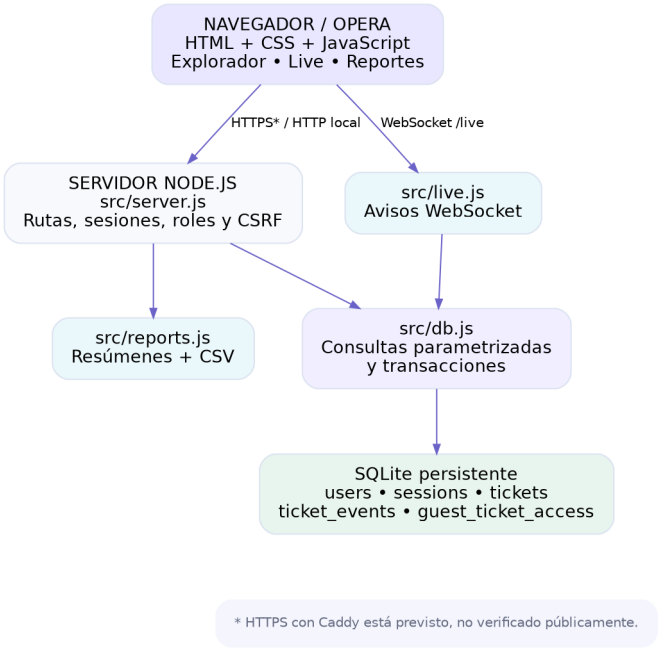


Figura 1. Arquitectura de componentes; la publicación HTTPS externa aún no ha sido validada.

2. Modelo de datos y relaciones

El módulo `src/db.js` crea cinco tablas, activa `PRAGMA foreign_keys=ON` y usa `PRAGMA journal_mode=WAL`. La migración desde v0.2 es aditiva: no elimina registros existentes.

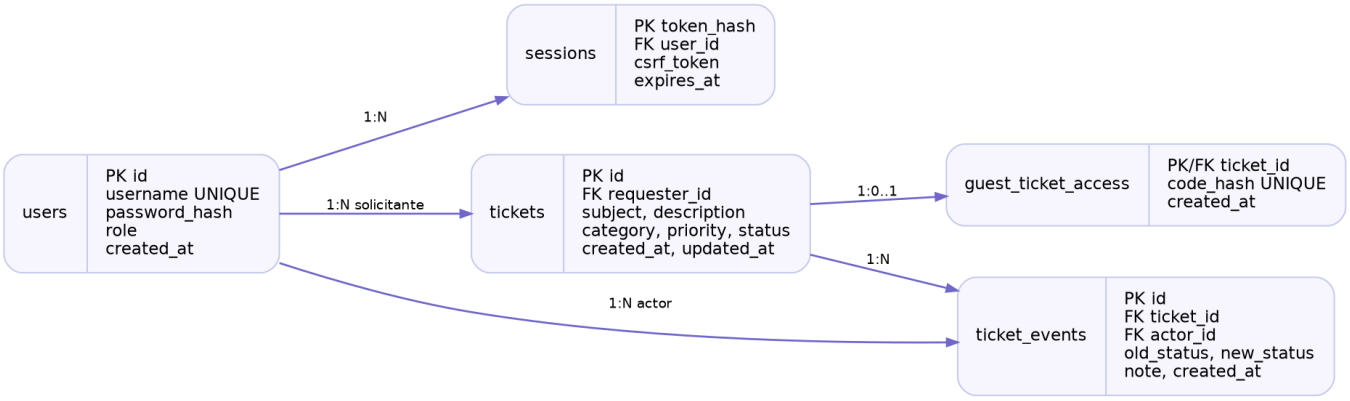


Figura 2. Modelo de cinco tablas basado en el esquema SQLite incluido en el código.

Tabla	Función / principales relaciones
users	Usuarios autenticados y cuenta técnica __invitado__; identificador PK y rol.
sessions	Sesiones vinculadas con users mediante user_id; token_hash y expiración.
tickets	Solicitudes vinculadas a requester_id de users; prioridad, categoría, estado y fechas.
ticket_events	Bitácora de tickets, actor_id a users, ticket_id a tickets, estados previo/nuevo y nota.
guest_ticket_access	Código privado hasheado, vinculado por ticket_id único a tickets.

Diccionario de campos detallado en Anexo D.

3. Procesos, endpoints y módulos de código



Figura 3. Secuencia de gestión desde el registro hasta seguimiento y reporte.

Endpoint	Método / autorización / finalidad
/login	GET/POST; público; formulario y autenticación con CSRF.
/	GET; autenticado; panel de tickets autorizados.
/tickets/new, /tickets	GET/POST; autenticado; crear solicitud.
/tickets/:id	GET; propietario o coordinador; detalle e historial.
/tickets/:id/status	POST; coordinador; cambiar estado.
/tickets/:id/notes	POST; coordinador; observación independiente.
/guest/new, /guest/tickets	GET/POST; público con CSRF; registrar invitado.
/seguimiento	GET/POST; público con código secreto; consultar invitado.
/reportes, /reportes.csv	GET; autenticado; métricas y exportación restringida.

Endpoint	Método / autorización / finalidad
/live	Upgrade WebSocket; autenticado, sujeto a origen y permisos.
/healthz	GET; público; verificación de vida.

Distribución del código

Archivo	Responsabilidad
src/server.js	Rutas, HTML, CSRF, roles y validaciones.
src/db.js	SQLite, migración, hashing, transacciones y consultas autorizadas.
src/live.js + public/live.js	Avisos WebSocket, recuperación HTTP y protección de borradores.
public/explorer.js	Búsqueda, filtros de estado, categoría y prioridad, y ordenamiento local durante Live.
src/reports.js	Agrupaciones y serialización segura de CSV.
public/styles.css	Diseño responsive y barras de reportes.
tests/	Pruebas automatizadas y regresión.
Dockerfile + compose.local.yml	Empaquetado y ejecución con volumen persistente.

4. Instalación local y variables de entorno

4.1. Requisitos

Descargue la fuente limpia, descomprímala en una ruta sin caracteres conflictivos, instale Node.js 22.13+ o 24 y abra PowerShell en la carpeta del proyecto. El proyecto no requiere instalar dependencias npm externas.

```
node --version
npm.cmd test
npm.cmd run check:agents
npm.cmd run check:codex
```

En una instalación nueva **y vacía**, cree cuentas de demostración una sola vez mediante `npm.cmd run seed`. Las contraseñas aparecen una sola vez en la terminal y deben custodiarse en privado. No vuelva a ejecutar seed para cambiar cuentas existentes.

```
$env:DATA_DIR = "D:\MesaHelp\Datos_Demo_Aislados"
$env:PORT = "3013"
npm.cmd run seed
npm.cmd start
```

Abrir en Opera `http://127.0.0.1:3013/login`. Con `DATA_DIR` separado no se toca la base original. En un despliegue público, `COOKIE_SECURE=1` requiere HTTPS correctamente configurado.

4.2. Entorno Docker local

```
docker compose -f compose.local.yml up -d --build
docker compose -f compose.local.yml ps
docker compose -f compose.local.yml logs --tail 80
```

El contenedor usa su propio volumen `ticket_data`. No comparte automáticamente la base SQLite del host. La guía admite funcionamiento local instalable; el funcionamiento del contenedor de esta nueva candidata debe

verificarse en Windows.

5. Seguridad y privacidad

- Contraseñas con scrypt más sal; no se guardan en texto claro.
- Sesiones basadas en token aleatorio cuya huella SHA-256 se almacena en sessions, cookie HttpOnly y SameSite=Lax; Secure se activa por configuración.
- Protección CSRF para los formularios POST, incluida la fase previa al login y los invitados.
- SQL parametrizado y verificación de permisos en servidor, no solo controles visuales.
- Cabeceras CSP, X-Frame-Options DENY, nosniff y Cache-Control no-store.
- Código privado de invitado hasheado; se muestra una sola vez; CSV sin descripciones, notas ni códigos.
- Límites por IP simples en memoria para intentos de login, registros invitados y consultas de seguimiento.

Alcance de la seguridad: estos controles son una base para el MVP, no constituyen auditoría integral. No hacer público el servidor con cuentas de demostración ni sin evaluar abuso, datos personales, respaldo, cookies bajo proxy, TLS y WebSocket.

6. Pruebas, respaldo y mantenimiento

Validación: 33/33 pruebas automatizadas aprobadas en Windows y GitHub Actions (main).
Cobertura: MVP, migración, Live, Explorador, reportes, CSV, prioridad y ordenamiento.
Los validadores Claude/Codex comprueban estructura, no ejecución real de agentes.
Pruebas manuales en Opera/Excel completadas. Instalación por un tercero: pendiente.

6.1. Copias de seguridad

```
$env:DATA_DIR = "D:\MesaHelp\Datos_Demo_Aislados"
$env:BACKUP_DIR = "D:\MesaHelp\Respaldos"
npm.cmd run backup
```

`scripts/backup-db.cjs` utiliza SQLite `VACUUM INTO` para producir un archivo consistente sin sobrescribir copias existentes. Para restaurar, detenga el servidor y copie una base respaldada a un **nuevo** DATA_DIR; pruebe allí antes de cualquier sustitución. No publicar SQLite, contraseñas ni sesiones.

6.2. Mantenimiento / problemas conocidos

- La base de datos no implementa asignación de responsables ni SLA medidos.
- La página de reportes no escucha Live; debe recargarse para cifras recientes.
- El canal WebSocket es mínimo, unidireccional y previsto para una instancia.
- No existe recuperación autónoma de contraseña ni adjuntos.
- El registro y seguimiento de tickets no demuestran que la incidencia fue resuelta realmente.
- El control de uso y copias de respaldo requiere un operador.

7. Repositorio, licencias y despliegue

Repositorio: https://github.com/KIRZONMAN/mesahelp-mvp (público; rama main).
Documentación: https://kirzonman.github.io/mesahelp-mvp/ (GitHub Pages).
No publicar data/, .env, respaldos, sesiones, secretos ni códigos privados.

Código declarado con licencia MIT en package.json. No se detectan dependencias npm externas. Revisar derechos de imágenes, fuentes y cualquier recurso incorporado al paquete final.

Estado de despliegue: MesaHelp fue validado con Node.js local en Windows. El código y la documentación están públicos en GitHub y GitHub Pages. Docker fue probado en una variante previa, no en esta última. El servidor Node.js no tiene despliegue web público.

Para actualizar una instalación, mantenga respaldo previo del código y SQLite, ejecute `npm test`, revise las diferencias de archivos y despliegue en una carpeta nueva. No sobrescriba directamente la instalación antigua mientras esté funcionando.